Autor: Iván García Santillán

Algoritmos predictivos de clasificación: Vecinos más cercanos (K-NN)

Introducción

El algoritmo k-Nearest Neighbors (k-NN) es un método de aprendizaje automático supervisado utilizado para problemas de clasificación y regresión. Éste tiene diversas aplicaciones prácticas en campos como el análisis de datos, aprendizaje automático y sistemas de recomendación.

En medicina, se utiliza para clasificar pacientes en categorías de riesgo basándose en características similares a casos conocidos.

En el sector financiero, ayuda en la detección de fraudes identificando patrones de transacciones sospechosas similares a casos previos de fraude.

En el comercio minorista y en servicios de *streaming*, k-NN es esencial para los sistemas de recomendación, sugiriendo productos, películas o música basándose en las preferencias de usuarios con gustos similares.

También es utilizado en la agricultura para la clasificación y diagnóstico de enfermedades de plantas, y en la robótica para la navegación y clasificación de obstáculos. La simplicidad y efectividad del algoritmo lo hacen versátil para aplicaciones que requieren clasificación o recomendación basada en similitudes de datos.

Aunque también se debe considerar algunas desventajas como una posible baja de la precisión comparado con otros algoritmos y un uso de memoria moderado dependiendo del tamaño del conjunto de datos utilizado.

A continuación, se detallan el funcionamiento del algoritmo y un ejemplo en Python, Keras y TensorFlow para la predicción del cáncer de mama utilizando un data set público.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Contenido

La idea fundamental detrás del algoritmo k-NN es que los puntos de datos similares tienden a estar cerca unos de otros en el espacio de características. Por lo tanto, para predecir una nueva instancia, el algoritmo busca los "k" puntos de datos más cercanos en el conjunto de entrenamiento y toma una decisión basada en la mayoría de las etiquetas de clase (en el caso de clasificación) o en el promedio (en el caso de regresión) de las etiquetas de los vecinos más cercanos.

Algunos conceptos clave relacionados con el algoritmo k-NN son los siguientes:

Parámetro k: el valor de "k" en k-NN representa el número de vecinos más cercanos que se deben considerar al hacer una predicción. Un valor pequeño de "k" (como 1) puede llevar a predicciones ruidosas y sensibles a valores atípicos, mientras que un valor grande de "k" suaviza las predicciones pero puede perder detalles finos en los datos. La elección del valor de "k" es crítica en k-NN y puede afectar significativamente el rendimiento del modelo.

Función de Distancia: para determinar qué puntos de datos son los más cercanos, se utiliza una función de distancia, como la distancia euclidiana, la distancia de Manhattan o la distancia de Minkowski. La elección de la función de distancia depende del problema y los datos.

Normalización: antes de aplicar k-NN, es importante normalizar las características para que todas tengan la misma influencia en la distancia entre puntos. Esto es especialmente importante cuando las características tienen escalas muy diferentes.

El algoritmo k-NN es simple de entender y de implementar, y es útil en una variedad de aplicaciones, como clasificación de documentos, recomendación de productos, diagnóstico médico y más. Sin embargo, es computacionalmente costoso en conjuntos de datos grandes, ya que requiere calcular la distancia entre la nueva instancia y todos los puntos de datos de entrenamiento.

A continuación, se presenta un ejemplo del algoritmo K-NN en Python, Keras y TensorFlow para la predicción del cáncer de mama (benigno=0, maligno=1)

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

utilizando el conjunto de datos Breast Cancer Wisconsin (Diagnostic) disponible en la librería scikit-learn.

```
# K-NN for breast cancer

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report
from sklearn.neighbors import KNeighborsClassifier

# Carga el conjunto de datos Breast Cancer
dataset = load_breast_cancer()
X = dataset.data # 569x30
y = dataset.target # 569x1

# Divide el conjunto de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normaliza los datos para que todas las características tengan una escala similar
scaler = MinMaxScaler(feature_range=(0,1)) # [0, 1]
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Crea y entrena el modelo K-NN
model = KNeighborsClassifier(n_neighbors=3, p=2, # Función euclidean weights='distance')

model.fit(X_train, y_train)

# Realiza predicciones usando el conjunto de prueba
y_pred = model.predict(X_test)

# Convierte las probabilidades en etiquetas binarias (0 o 1)
y_pred = (y_pred > 0.5)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

```
# Muestra el informe de evaluación del modelo entrenado
print(classification_report(y_test, y_pred))

# Matriz de confusión:
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
print("confusion matrix: \n", cm)
# gráfica de la CM
plt.figure(figsize = (8,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Prediction', fontsize = 12)
plt.ylabel('Real', fontsize = 12)
plt.show()

# Exactitud:
from sklearn.metrics import accuracy_score
acc = accuracy_score(y_test, y_pred)
print("accuracy: ", acc)

# Sensibilidad:
from sklearn.metrics import recall_score
recall = recall_score(y_test, y_pred)
print("recall: ", recall)

# Precisión:
from sklearn.metrics import precision_score
precision = precision_score(y_test, y_pred)
print("precision: ", precision)

# Especificidad
# 'specificity' is just a special case of 'recall'.
# specificity is the recall of the negative class
specificity = recall_score(y_test, y_pred, pos_label=0)
print("specificity: ", specificity)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

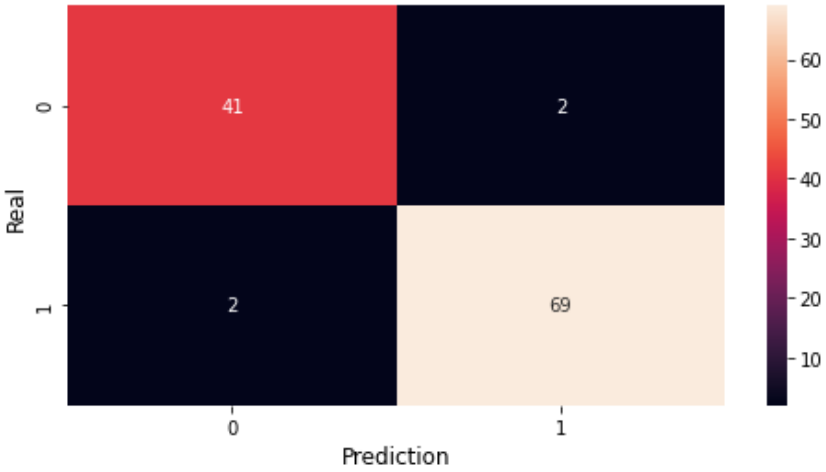
```
# Puntuación F1:
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_pred)
print("f1 score: ", f1)

# Área bajo la curva:
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y_test, y_pred)
print("auc: ", auc)

# Curva ROC
from sklearn.metrics import roc_curve
plt.figure()
lw = 2
plt.plot(roc_curve(y_test, y_pred)[0], roc_curve(y_test, y_pred)[1],
color='darkorange',lw=lw, label='ROC curve (area = %0.2f)' %auc)
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()

# R Score (R^2 coefficient of determination)
from sklearn.metrics import r2_score
R = r2_score(y_test, y_pred)
print("R2: ", R)
```

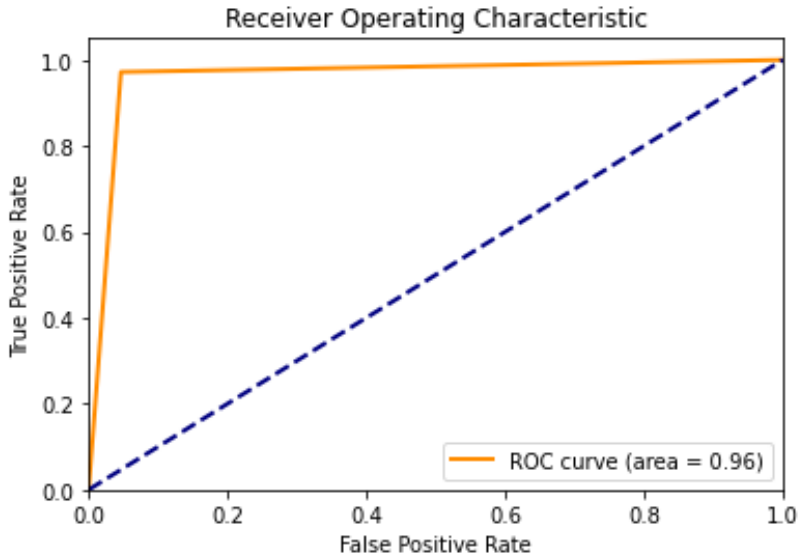
Los resultados del modelo K-NN se presentan a continuación:

```
accuracy:  0.96
recall:  0.97
precision:  0.97
specificity:  0.95
f1 score:  0.97
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

auc: 0.96
R2: 0.85





Un clasificador adecuado es aquel que alcanza una exactitud (*accuracy*) superior al 85 %. Por lo que, este modelo K-NN entrenado cumple con este criterio de calidad.

Adicionalmente, tenga en cuenta que se pueden ajustar varios parámetros del algoritmo (como *n_neighbors, p, weights*, etc.) para obtener un modelo óptimo.

Revise la documentación (ayuda en línea https://scikit-learn.org/stable/supervised_learning.html) del algoritmo para más detalles y experimente con ellos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Finalmente, aunque K-NN es un modelo fundamental en la caja de herramientas de análisis predictivo, como cualquier otra técnica, presenta tanto ventajas como desventajas que se resumen a continuación:

Ventajas de K-NN:

Fácil de entender e implementar: Una de las mayores ventajas de K-NN es su simplicidad. El algoritmo es intuitivo y fácil de implementar, y no requiere entrenamiento explícito, lo que lo convierte en un método perezoso (*lazy*).

Flexible a la función de la distancia: K-NN puede trabajar con cualquier función de distancia, lo que lo hace adecuado para contextos donde la noción de distancia es no estándar o muy específica (como distancia Euclidiana, Manhattan, Minkowski, o incluso medidas personalizadas).

Eficaz con un gran número de clases: K-NN puede ser eficaz cuando hay un gran número de posibles etiquetas de salida; su capacidad para diferenciar no está limitada por las características de los datos.

Desventajas de K-NN:

Alto costo computacional: K-NN puede ser muy lento en bases de datos grandes porque cada nueva predicción requiere un cálculo de distancia a cada uno de los puntos de entrenamiento, lo cual puede ser computacionalmente costoso.

Alto uso de memoria: dado que K-NN es un algoritmo perezoso (*lazy*) y no genera un modelo explícito, necesita almacenar todo el conjunto de datos de entrenamiento para hacer predicciones, lo que puede resultar en un alto uso de memoria.

Sensible a variables irrelevantes: K-NN puede ser muy sensible a características irrelevantes o redundantes porque todas las características contribuyen aproximadamente por igual a la distancia entre las muestras.

Sensible a datos no balanceados: en un conjunto de datos donde las clases no están equitativamente representadas, K-NN puede estar sesgado hacia las clases más frecuentes.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Dificultad con clases *overlapping*: K-NN puede tener dificultades cuando los límites de decisión entre clases son borrosos. Su desempeño puede degradarse significativamente si las clases se superponen considerablemente.

Efecto de la maldición de la dimensionalidad: a medida que el número de características aumenta, el espacio de características se vuelve exponencialmente grande, lo que diluye la efectividad de la distancia euclidiana y puede hacer que K-NN actúe de manera subóptima.

Conclusiones

En esta lectura se ha revisado los fundamentos del algoritmo k-Nearest Neighbors (k-NN) para abordar problemas de clasificación binaria utilizando Python y TensorFlow-Keras. K-NN es un algoritmo potente y flexible que se adapta bien a una variedad de problemas de clasificación y regresión, es crucial tener en cuenta sus limitaciones, especialmente en términos de eficiencia y rendimiento en grandes volúmenes de datos o datos con muchas características. A menudo, técnicas como la reducción de dimensiones, la selección de características, y el ajuste del número de vecinos (k) son necesarias para mejorar su eficacia y eficiencia.

Por lo que, se motiva a los estudiantes a revisar material y ejemplos adicionales disponibles libremente en Internet para profundizar en la comprensión del algoritmo y su aplicación eficaz en diferentes problemas de variada complejidad.

Bibliografía

Berzal, Fernando (2019). *Redes neuronales & Deep Learning*. USA: Independently published.

Gerón, A. (2023). *Aprende Machine Learning con Scikit-Learn, Keras y TensorFlow: Conceptos, herramientas y técnicas para conseguir sistemas inteligentes.* Tercera Edición. Anaya Multimedia.

Torres, J. (2020). *Python Deep Learning: Introducción práctica con Keras y TensorFlow 2*. Marcombo.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.